高级语言程序设计
实验报告

南开大学 计算机大类
姓名：王梓言
学号：2513900
班级:1009
2026 年 5 月 10 日

目录

根据高级语言程序设计大作业实验报告

根据高级语言程序设计大作业实验报告
一、作业题目
二、开发软件
三、课题要求
四、主要流程
整体流程
算法或公式
落子合法性与提子
气的判定
连通块收集与移除
打劫判定
MCTS 决策流程
终局判定
单元测试/功能验证设计
五、单元测试
六、收获
面向对象与规则引擎解耦
围棋规则细节实现
MCTS 与策略网络结合
测试驱动价值
工程化能力提升
七、AI 辅助开发说明
MCTS 算法模块中的 AI 辅助
SGF 棋谱解析模块中的 AI 辅助
Python 模型训练部分中的 AI 辅助
文档与测试说明中的 AI 辅助
AI 使用总结

一. 作业题目

围棋引擎系统设计与实现：基于面向对象的 19×19 围棋规则引擎、图形界面人机对弈、SGF 棋谱读写与 AI 决策模块（MCTS+神经网络评估）的综合开发。

二. 开发软件

1) 开发语言：C++17、Python 3
2) IDE/构建工具：CMake（建议配合 CLion / Visual Studio / Qt Creator）
3) GUI 框架：Qt6（Core / Widgets / Concurrent / Multimedia）
4) 其他模块：SGF 解析模块、数据提取工具、Python 模型训练与推理脚本

三. 课题要求

1) 面向对象：围棋棋盘、对局流程、AI 搜索节点等采用类封装。
2) 单元测试/功能验证：针对核心规则与关键流程进行案例验证。
3) 模型部分：集成 Python 评估器，为 MCTS 提供策略先验。
4) 验证：完成人机对局、棋谱加载回放、规则正确性验证（提子、自杀禁手、打劫、连续停着终局）。

四. 主要流程

1．整体流程

实现思路：

- 采用“核心规则层 + 对局控制层 + AI 搜索层 + UI 展示层 + 数据工具层”的分层架构。
- `Board`类负责围棋底层规则：合法性检查、连通块判断、气的计算、提子处理。
- `Game`类负责对局状态管理：当前玩家、落子历史、打劫判断、悔棋、停着终局流程。
- `MCTS`类负责 AI 决策：选择/扩展/模拟/回传，结合`PythonEvaluator`对策略概率排序。
- UI 层通过页面与棋盘组件展示局面，并可配合 SGF 模块实现复盘。

典型控制流程：

① 用户/AI 输入落子 → ② `Game::playMove`发起规则判断 → ③ `Board::playMove`执行落子与提子
→ ④ 写入历史并切换行棋方 → ⑤ AI 阶段由 MCTS 搜索给出最佳下一手

→ ⑥ 对局结束后可导出 SGF 或进入复盘流程。

2．算法或公式

以下以 x,y 表示坐标，`color`表示当前落子方。

（1）落子合法性与提子（Board::playMove）

- 若越界或目标点非空，则非法。
- 先试下该子，再检查四邻对方棋块是否无气；若无气则整块移除（提子）。
- 若未产生提子且己方新棋块无气，则判定为自杀手，回滚并返回非法。

（2）气的判定（Board::hasLiberty）

- 对同色连通块执行 DFS：
  - 若搜索过程中遇到空点，说明有气，返回真；
  - 连通块全部遍历后仍无空邻点，则无气。

（3）连通块收集与移除

- `collectGroup`用于收集同色连通块（可用于扩展分析/标注）。
- `removeGroup`递归移除整个被提棋块并统计提子数。

（4）打劫判定（Game::isKo）

- 对候选着法在临时棋盘试下；
- 若试下后局面与“前两手记录的棋盘局面”相同，则视为打劫禁入点，禁止该手。

（5）MCTS 决策流程

- Selection：沿 PUCT 值最大的子节点向下选择；
- Expansion：从未尝试着法中扩展一个新节点；
- Simulation：调用轻量评估函数（地盘近似+贴目）得到局面得分；
- Backpropagation：将结果回传至祖先节点，更新访问次数与累计收益。
- 若接入 Python 评估器，根节点着法先按策略概率排序并截断 Top-K，提高搜索效率

（6）终局判定

- 当历史记录最后两手均为`pass`时，判定对局终止。

3．单元测试/功能验证设计

针对围棋核心规则与流程，分别设计用例：

- 空棋盘首手合法性；
- 禁止落在已有棋子位置；
- 提子正确性（单子提取、连块提取）；
- 自杀禁手正确性；

- 打劫限制有效性；
- 悔棋后棋盘恢复正确；
- 双方连续停着后终局识别；
- SGF 读写一致性（可解析、可回放）。

五. 单元测试

测试案例定义（示例）

表 1：测试案例

1) 输入：黑(3,3)
   预期：落子成功，当前行棋方切换为白。
   目的：基本落子流程。
2) 输入：同一点重复落子
   预期：第二手返回非法。
   目的：点占用合法性校验。
3) 输入：构造包围后提白一子
   预期：白子被移除，对应点变为空。
   目的：提子逻辑验证
4) 输入：构造无气自杀手
   预期：返回非法，棋盘不变化。
   目的：自杀禁手验证。
5) 输入：经典打劫型，立即提回
   预期：提回手被禁止。
   目的：打劫判定验证。
6) 输入：连续两手 pass
   预期：`isTerminal`为真（终局）。
   目的：终局识别。
7) 输入：落子后执行 undo
   预期：棋盘恢复到上一步，行棋方恢复正确。
   目的：悔棋功能验证。

测试结果说明：

- 在规则联调阶段，曾出现“打劫测试用例预期值错误”导致误判失败；修正测试期望后通过。
- AI 决策模块在未接入神经网络时可正常返回落子；接入策略先验后，搜索收敛

速度与走子合理性有提升。

六. 收获

1．面向对象与规则引擎解耦

- 将棋盘规则（Board）与对局流程（Game）解耦后，规则可复用于 UI、AI、数据提取多个模块，降低耦合度并提升可维护性。

2．围棋规则细节实现

- “先落子再判气再提子”的顺序必须严格；
- 自杀手、打劫、停着终局是最容易遗漏的边界条件；
- 历史局面记录对于悔棋、复盘、规则判定都非常关键。

3．MCTS 与策略网络结合

- 纯 MCTS 可工作，但在 19 路盘面分支过大；
- 策略网络用于根节点着法排序与裁剪能显著减少无效搜索。

4．测试驱动价值

- 规则代码常因边界条件导致“看似可下却不合法”；
- 固定测试案例可快速回归验证，能在重构后第一时间发现问题。

5．工程化能力提升

- 通过 CMake 组织多模块工程、Qt 构建 GUI、Python 训练推理联动，形成了“规则-界面-AI-数据”闭环开发流程，为后续加入更强评估模型和更完整数子规则打下基础。

七、AI 辅助开发说明

在本次“围棋引擎系统设计与实现”项目开发过程中，我在部分模块的设计、调试和文档整理中使用了 AI 工具进行辅助（主要是 ChatGPT 4 以及 Gemini）。AI 的作用主要体现在提供开发思路、辅助分析代码结构、提示可能遗漏的边界情况以及帮助整理说明文档等方面，核心规则实现、模块整合、测试验证和最终调试仍由本人完成。

1．MCTS 算法模块中的 AI 辅助

在 AI 决策模块中，我使用 AI 辅助理解和梳理了 MCTS（蒙特卡洛树搜索）的基本流程，包括 Selection、Expansion、Simulation 和 Backpropagation 四个阶段。帮助我明确了 MCTS 节点中需要维护的信息，例如访问次数、累计收益、父节点、子节点以及候选着法等。同时，在设计搜索流程时，AI 也辅助我分析了如何结合策略先验对候选落子进行排序和裁剪。不过，具体的代码组织、类之间的关系、与 Game 和 Board 模块的接口连接，以及实际运行中的调试

和修改，均由本人根据项目需求完成。

2．SGF 棋谱解析模块中的 AI 辅助

在 SGF 棋谱解析模块中，我使用 AI 辅助理解 SGF 文件的基本格式，例如节点、属性、落子坐标、分支结构以及棋谱回放流程。

AI 在这一部分主要起到了资料整理和思路提示的作用，帮助我明确了 SGF 解析时需要关注的关键内容，包括：

- 如何读取黑白双方的落子信息；
- 如何将 SGF 坐标转换为棋盘坐标；
- 如何按照棋谱顺序进行回放；
- 如何处理部分棋谱中的分支结构；
- 如何设计 SGFParser 与 Game 模块之间的接口。

在实际实现中，我根据项目结构编写了解析逻辑，并结合围棋棋盘模块进行了回放测试，确保棋谱读取后能够正确还原对局过程。

3．Python 模型训练部分中的 AI 辅助

在少部分 Python 模型训练与评估模块中，我使用 AI 辅助梳理了模型训练脚本的基本流程，包括数据读取、特征构造、模型训练、保存模型以及在 C++ 端调用评估结果的整体思路。